

试卷二维码打包格式规范

版本：v1（2026-08-02 定稿） 约束双方：印刷系统（生成）与扫码/分类端（解析）。

一. 概述

本格式把试卷标识的全部字段压进一个 56 bit 整数，QR 用 numeric 模式编码，任意纠错等级下 v1 (21×21) 即可装下。

二. 载荷内容与位布局

1. 载荷

QR 码内容 固定 17 位十进制数字（无分隔符、无后缀，左补零）。如：00284518902665574

2. 位布局

其二进制按 MSB → LSB（读取顺序即字段顺序）布局，红框内为五元组（43 bit，bit 5-47）：

位置 1bits 份数 7bits 学校 13bits 类型 1bits 班号区 19bits 试卷编号 10bits 页码 5bits

位段	字段	位宽	范围	说明
bit 55	位置	1	0-1	QR 在页面上的位置：0=右上，1=左下
bit 48-54	份数	7	0-127	学生份数 = studentid
bit 35-47	学校	13	0-8191	
bit 34	班级类型	1	0-1	0=兴趣班，1=行政班
bit 15-33	班号区	19	—	按班级类型分支解读（见下）
bit 5-14	试卷编号	10	0-1023	
bit 0-4	页码	5	0-31	物理页码（非扫描序号）
bit 5-47	五元组	43	—	学校+班级类型+班号区+试卷编号（规则见 二.4）
合计		56		

班号区（bit 15-33，共 19 bit）按班级类型分支解读：

行政班（bit34=1）：

位段	字段	位宽	范围	说明
bit 27-33	级数	7	0-127	该级一年级入学年份的后两位（如 2026 级 → 26）
bit 20-26	班号	7	0-127	
bit 15-19	科目	5	0-31（码表见 二.3）	

兴趣班（bit34=0）：

位段	字段	位宽	范围	说明
bit 26-33	班号	8	0-255	
bit 15-25	科目	11	0-2047	

3. 码表

班级类型（1 bit）

值	含义
0	兴趣班
1	行政班

科目码表（行政班，5 bit）

值	科目	值	科目
0	语文	10	信息科技
1	数学	11	通用技术
2	英语	12	体育与健康
3	物理	13	音乐
4	化学	14	美术
5	生物	15	心理健康
6	政治（道德与法治）	16	劳动
7	历史	17	综合实践
8	地理	18	日语
9	科学	19	俄语
		20-30	预留
		31	作文

4. 五元组规则（unique_id，bit 5-47）

"学校+班级类型+(级数)+班号+科目+试卷编号"五元组唯一定位一份试卷， 共 43 bit（bit 5-47）。

业务上做主键/查询时按整体 43 bit 使用；需要按维度筛选时按子字段位段提取。 注意五元组包含班级类型位，因此同一学校下行政班与兴趣班的五元组 永不冲突。

三. 样例

1. 行政班



QR 内容: 00284518902665574

二进制:

0 0000001 00000010111000 1 0010111 0000010 00000 0010001011 00110

字段	位段二进制	十进制	含义
位置	0	0	右上
份数	0000001	1	第 1 份 (学生 1 号)
学校	00000010111000	88	
班级类型	1	1	行政班
级数	0010111	23	一年级入学于 2023 年 (取后两位 23)
班号	0000010	2	2 班
科目	00000	0	语文 (码表见 二.3)
试卷编号	0010001011	139	
页码	00110	6	第 6 页
五元组	0000 ... 1011	95122686091	五元组拼接 (bit 5-47, 中间省略)

2. 兴趣班



QR 内容：63897844176979715

二进制：

1 1100011 0000001011000 0 00001100 00000101010 0000111000 00011

字段	位段二进制	十进制	含义
位置	1	1	左下
份数	1100011	99	第 99 份
学校	0000001011000	88	
班级类型	0	0	兴趣班
班号	00001100	12	兴趣班 12 号 (bit 26-33, 全 8 位为班号)
科目	00000101010	42	兴趣班科目码 (bit 15-25)
试卷编号	0000111000	56	
页码	00011	3	第 3 页
五元组	0000...1000	94514489400	五元组拼接 (bit 5-47, 中间省略)

四. 版本协商

本版本的判别特征：QR 内容为 17 位纯数字（v1 维数在 numeric 模式下的 极限长度，当前 56 bit 字段布局恰好用满）。由于容量已满，任何新增字段都必然导致位数膨胀、进入更高维数——届时新版本自然以新的位数特征与本版本区分。是否在后续版本中引入显式版本号字段，留待扩展时酌情决定；本版本不内嵌版本号，解析端按"QR 内容的位数形态"分派解析器即可。

五. 解码样例

注意：56 bit 超出 32 位整数范围——JS 必须用 BigInt，Java 用 long。

- 1. 只取五元组（unique_id, 43 bit, bit 5-47）

业务上最常用的操作：不解析全部字段，只提取 43 bit 的五元组（bit 5-47 = 学校+班级类型+级数+班号+科目+试卷编号）。

```
// JavaScript
function uniqueIdOf(s) {
  if (!/^\d{17}$/.test(s)) throw new Error("not a packed qr");
  return Number((BigInt(s) >> 5n) & ((1n << 43n) - 1n));
}
uniqueIdOf("00284518902665574"); // 95122686091
```

```
// Java
static long uniqueIdOf(String s) {
  return (Long.parseLong(s) >>> 5) & ((1L << 43) - 1);
}
uniqueIdOf("00284518902665574"); // 95122686091
```

```
# Python
def unique_id_of(s: str) -> int:
  return (int(s) >> 5) & ((1 << 43) - 1)
unique_id_of("00284518902665574") # 95122686091
```

2. 全字段解析

JavaScript / TypeScript

```
function parseQR(s) {
  if (!/^\d{17}$/.test(s)) throw new Error("not a packed qr");
  const v = BigInt(s);
  if (v >= 1n << 56n) throw new Error("overflow");

  const position = Number((v >> 55n) & 1n);
  const copy     = Number((v >> 48n) & 0x7Fn);
  const school   = Number((v >> 35n) & 0x1FFFn);
  const classType = Number((v >> 34n) & 1n);
  const region   = Number((v >> 15n) & 0x7FFFFn); // 班号区 19 bit
  const paperId  = Number((v >> 5n) & 0x3FFn);
  const page     = Number(v & 0x1Fn);

  let grade = 0, classNo, subject;
  if (classType === 1) { // 行政班: 级数7 | 班号7 | 科目5
    grade = (region >> 12) & 0x7F;
    classNo = (region >> 5) & 0x7F;
    subject = region & 0x1F;
  } else { // 兴趣班: 班号8 | 科目11
    classNo = (region >> 11) & 0xFF;
    subject = (region & 0x7FF) & 0x7FF;
  }
}
```

```

        subject = region          & 0x7FF;
    }
    return { position, copy, school, classType, grade, classNo, subject, paperId,
page };
}

// 用例
console.log(parseQR("00284518902665574"));
// { position: 0, copy: 1, school: 88, classType: 1,
//   grade: 23, classNo: 2, subject: 0, paperId: 139, page: 6 }

console.log(parseQR("63897844176979715"));
// { position: 1, copy: 99, school: 88, classType: 0,
//   grade: 0, classNo: 12, subject: 42, paperId: 56, page: 3 }

```

Java

```

public record QrFields(int position, int copy, int school, int classType,
                      int grade, int classNo, int subject,
                      int paperId, int page) {

    public static QrFields parse(String s) {
        if (!s.matches("\\d{17}")) throw new IllegalArgumentException("not a
packed qr");
        long v = Long.parseLong(s);           // 56 bit < 2^63, 无需 unsigned
        if (v >>> 56 != 0) throw new IllegalArgumentException("overflow");

        int position = (int) (v >>> 55);
        int copy      = (int) (v >>> 48) & 0x7F;
        int school    = (int) (v >>> 35) & 0x1FFF;
        int classType = (int) (v >>> 34) & 1;
        int region    = (int) (v >>> 15) & 0x7FFFF; // 班号区 19 bit
        int paperId   = (int) (v >>> 5)  & 0x3FF;
        int page      = (int) v & 0x1F;

        int grade = 0, classNo, subject;
        if (classType == 1) { // 行政班: 级数7 | 班号7 | 科目5
            grade = (region >>> 12) & 0x7F;
            classNo = (region >>> 5) & 0x7F;
            subject = region & 0x1F;
        } else { // 兴趣班: 班号8 | 科目11
            classNo = (region >>> 11) & 0xFF;
            subject = region & 0x7FF;
        }
        return new QrFields(position, copy, school, classType,
                             grade, classNo, subject, paperId, page);
    }
}

// 用例
QrFields f = QrFields.parse("00284518902665574");

```

```
// QrFields[position=0, copy=1, school=88, classType=1,  
//           grade=23, classNo=2, subject=0, paperId=139, page=6]  
  
QrFields g = QrFields.parse("63897844176979715");  
// QrFields[position=1, copy=99, school=88, classType=0,  
//           grade=0, classNo=12, subject=42, paperId=56, page=3]
```